SUBJECT VOCABULARY

initialise to set variables to their starting values at the beginning of a program or subprogram

initialisation the process of assigning an initial value to a variable

assignment statement the SET…TO command is used to initialise variables in pseudocode, for example:

```
SET anotherGo TO 0
SET correct TO False
```

You can put an initial value into a variable by:

- initialising it when the program is run (e.g. `SET total TO 0`, in pseudocode)
- reading a value from a keyboard or other device (e.g. `RECEIVE admissionCharge FROM (INTEGER) KEYBOARD`).

Once a variable has been **initialised** an **assignment statement** is used to change its value (e.g. `SET total TO total + admissionCharge`).

In Python this would be:

```
total = 0
total = total + admissionCharge
```

In Java this would be:

```
Scanner scan = new Scanner(System.in);
int admissionCharge = scan.nextInt();
int total = 0;
total = total + admissionCharge;
```

In C#, variables must be declared before use. When you declare a variable, you need to state the data type that the variable will store, for example:

```
// declare a variable called total that will be used to store floating point numbers and assign the value 0.0 to it
float total = 0.0;
// add the value stored in the variable admissionsCharge to the total
total = total + admissionsCharge;
```

GENERAL VOCABULARY

intended designed for a certain purpose

If a variable, such as a loop counter, is **intended** to hold a running total, then it should always be initialised to a starting value. Some programming languages won't execute if the programmer fails to do this; others will do so but may well produce some unexpected results.

SKILLS CRITICAL THINKING, DECISION MAKING

ACTIVITY 1

DATA TYPES

1 Investigate what data types are available in the high-level language you are studying. Produce a table similar to Table 2.1 to give a summary of your findings.

2 What do you think is an appropriate data type for each of these items?
   a the test score of an individual learner
   b the average score for a group of learners
   c whether or not the pass mark for the test has been achieved.

3 Look back over the algorithms you wrote in Unit 1 and find instances of variable initialisation.

SUBJECT VOCABULARY

type coercion the process of converting the value stored in a variable from one data type to another

TYPE COERCION

Sometimes the data type of a variable gets changed during program execution. This is known as **type coercion**. For example, if an integer value and a real value are used in an arithmetic operation, the result will always be a real.

In Python, type coercion is done automatically, for example, the output from the following program is 3.25.

```
x = 1
y = 2.25
z = x + y
print(z)
```

Type coercion is also automatic in Java:

```
int x = 1;
double y = 2.25;
double z = x + y;
System.out.print(z);
```

In C# this is generally referred to as casting. As with Python this can be automatic for many data type conversions and is known as implicit casting. For more complicated conversions you may need to research explicit casting.

One frequently used type of explicit casting is converting from a string to a numeric value. This is used when getting input from a user as `Console.Readline`. For example, the following C# code will output 3.25 and demonstrates automatic type coercion.

```
int x;                  // declares a variable called x that will store integers
double y, z;            // declares two variables called y and z that will store
                        floating point numbers
x = 1;
y = 2.25;
z = x + y;
Console.WriteLine(z);
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING, DECISION MAKING

ACTIVITY 2

MONITORING VISITOR NUMBERS



A theme park uses a program to monitor the number of people entering and exiting the park. The maximum number of visitors at any one time must not exceed 10 000. When the number of people in the park reaches the maximum, a 'Park Full' message is displayed at the entrance gate. Children can visit the park free of charge. Adults must pay AED 125 admission. The program records the amount of money collected at the gate.

1 What are the variables needed in the program?

2 Select an appropriate data type for each variable and constant.

9 INPUT/OUTPUT

All program data has to be entered (input) and information is output. The input could be automatic, e.g. from a sensor, but is often provided by human users. It is essential that this input is checked to ensure that it is what could reasonably be expected and falls within a certain range. Incorrect input could cause problems or even catastrophes.

LEARNING OBJECTIVES

- Explain the need for validation
- Write code that validates user input
- Write code that accepts and responds appropriately to user input
- Write code that reads to and writes from a text file

USER INPUT

Most programs require some form of input either from a user or from a file. You already know how to receive user input from a keyboard.

A program can be made much more 'user friendly' by displaying helpful messages informing users of what they are expected to enter and confirming successful input.

VALIDATION

It is important to ensure that data entered by the user is valid, as invalid data can cause a program to behave unexpectedly or even stop altogether. If the data entered into a program is incorrect, the output it produces will also be wrong. This is sometimes called the Garbage In, Garbage Out (GIGO) principle.

SUBJECT VOCABULARY

validation to check that the data entered by a user or from a file meets specified requirements

Any program that requires data entry should have appropriate forms of **validation** built in. But validation can't guarantee that the data entered is correct. It can only make sure that it is reasonable.

There are a number of different types of validation.

RANGE CHECK

A range check is used to ensure that the data entered is within a specified range. Study this algorithm, written in Pearson Edexcel pseudocode which checks that the number entered is between 1 and 10.

```
BOOLEAN valid
SET validNum TO False
WHILE validNum = False DO
   SEND 'Please enter a number between 1 and 10:' TO
   DISPLAY
   RECEIVE number FROM (INTEGER) KEYBOARD
   IF number >= 1 AND number <= 10 THEN
      SET validNum TO True
   END IF
END WHILE
SEND 'You have entered:' & number TO DISPLAY
```

The algorithm uses a Boolean variable named `validNum` as a status flag. It is initially set to False. The `WHILE` loop continues running until `validNum` is equal to True. An `IF` statement determines if the value of `validNum` should be set to True.

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 20

IMPLEMENTING RANGE CHECKS

Implement the range check algorithm in the high-level programming language you are studying.



PRESENCE CHECK

Another type of validation is a presence check. This simply ensures that a value has been entered, preventing the user from leaving an input blank.

This algorithm asks the user to input their name and uses a presence check to ensure they have entered a value. Any value will cause the loop to finish. It will keep asking the user to input their name until they input a value.

```
SET userName TO ' '
WHILE userName = ' ' DO
    RECEIVE userName FROM (STRING) KEYBOARD
END WHILE
SEND 'Hello' & userName TO DISPLAY
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 21

IMPLEMENTING A PRESENCE CHECK

Implement the presence check algorithm in the high-level programming language you are studying.

GENERAL VOCABULARY

predefined already defined before the start of something

LOOK-UP CHECK

A look-up check is used to test that a value is one of a **predefined** set of acceptable values. The list of acceptable values can be stored in a one-dimensional array.

This algorithm stores a list of valid form names in an array. It compares the form name entered by the user with the values in the array.

```
SET arrayForms TO ['7AXB', '7PDB', '7ARL', '7JEH']
RECEIVE form FROM (STRING) KEYBOARD
SET valid TO False
SET index TO 0
SET length TO LENGTH(arrayForms)
WHILE valid = False AND index < length DO
    IF form = arrayForms[index] THEN
        SET valid TO True
    END IF
    SET index TO index + 1
END WHILE
```

```
IF valid = True THEN
    SEND 'Valid form' TO DISPLAY
ELSE
    SEND 'The form you entered does not exist.' TO DISPLAY
END IF
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 22

IMPLEMENTING A LOOK-UP CHECK

Implement the look-up check algorithm in the high-level programming language you are studying.

LENGTH CHECK

It is sometimes necessary to check that the length of a value entered falls within a specified range. For example, all UK postcodes are between six and eight characters long, so validation could be used to check that the length of a postcode entered is within this range. Needless to say, that doesn't necessarily mean it is correct.

```
RECEIVE postCode FROM (STRING) KEYBOARD
SET length TO LENGTH(postCode)
IF length >= 6 AND length <= 8 THEN
    SEND 'Valid' TO DISPLAY
ELSE
    SEND 'Invalid' TO DISPLAY
END IF
```

SKILLS CRITICAL THINKING

ACTIVITY 23

IMPLEMENTING A LENGTH CHECK

Implement the length check algorithm in the high-level programming language you are studying.

TESTING VALIDATION RULES

It is important to test your validation rules to ensure they work as expected. You should use:

Normal data – This is data that is within the limits of what should be accepted by the program. For example, a password with seven characters fulfils the validation rule that states that passwords must be between six and eight characters in length.

Boundary data – This is data that is at the outer limits of what should be accepted by the program. For example, if a validation rule specifies that the range of acceptable values is >= 75 AND <= 100, then a value of 100 is at the upper limit and a value of 75 at the lower.